

Ahsanullah University of Science & Technology
Department of Computer Science & Engineering

Real Madrid Museum — Hall of Legends

An Interactive 3D Museum built with Three.js / WebGL

Computer Graphics Lab (CSE 4204)

Final Project Report

Submitted By

Name	Student ID
Syed Abir Hossain	20220104013
Alim Bin Yeasin	20220104016

0. Project Overview

This report documents **Real Madrid Museum — Hall of Legends**, a browser-based, first-person interactive 3D museum built for the Computer Graphics Lab (CSE 4204) project. The application is written from scratch in JavaScript on top of **Three.js / WebGL**, and lets a visitor walk through a virtual exhibition hall containing four procedurally modelled trophies, a re-posed statue of Cristiano Ronaldo built from real glTF rigs, waving cloth banners driven by a custom vertex shader, wall posters and a historical “magazine” overlay describing every European Cup final, Ballon d’Or win and domestic honour the club has won.

The codebase is organised as ES modules under `src/` — `world/` (room, exhibits, trophies, statue, flags, textures), `player/` (first-person/eagle-eye camera and movement), and `ui/` (the magazine overlay and its content) — orchestrated by `main.js`.

1. Problem Analysis

1.1 Project Requirements

The primary goal was to render and let a user freely explore a themed 3D exhibition hall using Three.js, while covering the core computer-graphics topics of the course: custom shaders, dynamic lighting, perspective projection, texturing, keyframe/procedural animation and interactive controls. Concretely, the project set out to:

- Build a fully enclosed 3D gallery (floor, walls, ceiling, columns, entrance) with realistic scale ($40 \times 26 \times 6$ world units) and PBR materials.
- Model every trophy and prop as real geometry (lathed/extruded meshes) rather than using flat sprites or stock assets, so lighting and shadows behave correctly on them.
- Import, re-rig and re-pose a real skinned humanoid glTF model (Mixamo “Xbot”) plus a photo-scanned head model into a single Cristiano Ronaldo statue.
- Implement first-person WASD/mouse-look navigation with physically plausible collision against walls, columns and exhibits.
- Provide a second, eagle-eye orbit camera that shows an animated third-person avatar walking around the same room.
- Add interactive exhibits: proximity/line-of-sight detection, an on-screen prompt, and a paginated magazine UI that opens historical content per exhibit.
- Write at least one hand-written GLSL shader (the cloth-animation vertex shader used for the club banners) rather than relying only on built-in materials.
- Keep the experience resilient — e.g. the scene must not break if the Wikimedia photo textures fail to load, or if the browser denies the Pointer Lock API.

1.2 Core Integration

Models, camera, lighting and texturing are integrated through a small set of composable builder functions that all return a `THREE.Group` plus metadata, which `main.js` then wires into one scene graph:

- **Scene assembly** — `main.js` calls `buildMuseum()`, `buildUCLExhibit()`, `buildDomesticExhibit()`, `buildStatue()`, `buildPainting()` and `buildFlags()` in sequence, each anchored to a fixed world-space position taken from `museum.anchors`, then adds their groups to a single `THREE.Scene`.

- **Texturing** — a single `TextureLibrary` class (`src/world/textures.js`) is instantiated once and injected into every builder. It draws every surface (marble floor, wall panel, ceiling coffer, carpet, wood grain, gold plaques, poster art, the club crest) procedurally on an off-screen `<canvas>`, wraps it in a `THREE.CanvasTexture`, and — once `assets/manifest.json` resolves — paints a real Wikimedia Commons photograph on top of the same canvas and calls `texture.needsUpdate = true`. This means the museum is fully textured and usable even before (or without) the network photos, and photos simply “upgrade” the same materials in place.
- **Camera** — a single `THREE.PerspectiveCamera` (72° FOV, near 0.1 / far 90) is shared by both view modes; `PlayerControls` repositions and re-aims it every frame instead of swapping cameras, and cross-fades FOV/position between the first-person eye point and the eagle-eye orbit target.
- **Lighting** — layered lighting keeps the room from looking flat: a cool `HemisphereLight` + low-level `AmbientLight` wash the whole hall, eight ceiling `SpotLights` give an even gallery glow, warm `PointLight` sconces flank the entrance, and every exhibit (UCL wall, domestic wall, the statue, the painting) has its own dedicated, independently animated accent spotlight so it reads as a deliberately lit museum piece rather than uniform ambient fill.
- **Reflections** — a tiny offscreen “environment scene” of coloured boxes (a cool back wall, two warm key panels, a gold floor-bounce panel) is baked once through `THREE.PMREMGGenerator` and assigned to `scene.environment`, giving the gold/silver trophy and trim materials believable studio-style reflections without a real HDRI file.
- **Collision integration** — rather than hand-authoring bounding boxes, any mesh/group whose `userData.solid` flag is set has its world-space `THREE.Box3` computed automatically from its geometry after the scene graph settles (`solidColliders()`), so pedestals, walls and columns can never silently drift out of sync with their visual mesh.

1.3 Technical Challenges

Several parts of the project required non-trivial applied mathematics beyond calling built-in Three.js helpers:

- **View-direction vectors from yaw/pitch** — the camera's forward vector is derived analytically each frame as $(-\sin(\text{yaw})\cos(\text{pitch}), \sin(\text{pitch}), -\cos(\text{yaw})\cos(\text{pitch}))$, a standard spherical-to-Cartesian conversion, and is reused both for movement (project onto the XZ plane) and for the interaction raycast.
- **Camera-relative movement** — keyboard input (forward/back/strafe) is rotated into world space with a 2-D rotation using $\sin(\text{yaw})/\cos(\text{yaw})$, then smoothed toward the target velocity with exponential damping `lerp(v, target, 1 - e-12·dt)` so movement never feels frame-rate dependent or twitchy.
- **Axis-separated collision resolution** — the player is moved and clamped one axis at a time (not by pushing out along the nearest face), because resolving both axes together was found to teleport the player sideways at box corners; a second correction pass then re-checks both axes, and a separate “unstuck” routine eases the player out of the single deepest-penetrated box at a capped walking speed if they ever end up inside one.
- **Dual-camera blending** — first-person and eagle-eye share one camera object. A blend factor is eased with `smoothstep(x) = x2(3-2x)` each frame, and the camera position/look-at target/FOV are all linearly interpolated between the first-person eye point and an orbiting third-person point, so the transition between modes is continuous rather than a hard cut.

- **Orbit-camera trigonometry** — the eagle-eye camera position is computed as a point on a sphere of radius `dist` around the avatar's head: $x = \text{target.x} + \sin(\text{yaw}) \cdot \cos(\text{pitch}) \cdot \text{dist}$, with the equivalent `y/z` terms, then clamped back inside the room bounds so the camera can never end up outside a wall.
- **Cloth simulation shader** — the banners' vertex shader sums two travelling sine waves in UV-space, scales their amplitude by $(1-v)^{1.35}$ (a slack term so the cloth is pinned near the rod and freest at the hem) and by horizontal distance from the centre line, then displaces each vertex along its local normal. The corresponding partial derivatives of that same expression are computed analytically to reconstruct a per-vertex normal (rather than only recomputing normals from the deformed mesh), so the specular highlight ripples correctly with the fabric instead of just its silhouette.
- **Skeletal re-posing via quaternions** — the statue's body is a rigged Mixamo model whose skeleton is driven by two small helper functions, `aim()` and `twist()`, that read a bone's and its child's world-space position from their `matrixWorld`, compute the rotation needed to align the bone with a target direction, and premultiply it onto the bone's local quaternion — effectively solving a per-joint orientation problem to bend the default T/A-pose into the statue's “Siuu” landing stance.
- **Procedural kit geometry on a moving skeleton** — the statue's shorts, sleeves and boots are generated as tube/ribbon meshes whose control points are the interpolated (`lerp`) world-space positions between pairs of joints, so the carved “clothing” conforms to the re-posed skeleton instead of being modelled once in a fixed pose.
- **Lathed and swept trophy profiles** — each trophy body is a `THREE.LatheGeometry` revolved from a hand-tuned 2-D radius/height profile matched to the real trophy's silhouette, while the handles are flat metal ribbons extruded along a `CatmullRomCurve3` using the curve's normal/binormal frame, so they read as flat scrollwork rather than round tubes.
- **Frame-rate independent simulation** — the render loop measures true elapsed time with `THREE.Clock` but clamps any delta above 120 ms (a tab switch, a GC pause, texture decoding) down to a fixed 1/60 s step, preventing the player from being catapulted across the room after the browser stalls.

2. User Interaction

2.1 Key Interaction

Keyboard state is tracked in a Set updated on keydown/keyup, decoupled from any single “once-per-press” event so multiple keys can be held simultaneously:

- `W / A / S / D` or the arrow keys drive forward/back/strafe. Held keys are normalised into a (`fwd`, `strafe`) pair each frame, rotated by the player's current yaw into world space, and normalised so diagonal movement is not faster than cardinal movement.
- `Shift` enables sprinting, raising the target speed from 4.4 to 7.2 world-units/second.
- `E` (or a left click while looking at an exhibit) activates whichever exhibit the player is currently facing/near — opening the UCL, Ballon d’Or or Domestic Honours magazine, or toggling the Bernabéu painting between its old and new photographs.
- `V` (or the on-screen button) switches between first-person and eagle-eye camera modes.
- `Esc` releases the pointer lock and hands control back to the OS cursor; releasing the lock never disables the controls — drag-to-look and the movement keys keep working regardless.

- Inside the magazine overlay, the left/right arrow keys (or `h/l`) turn pages and `E/Esc` closes it; these keys are consumed by the overlay so the same keypress can never “leak through” and immediately re-trigger the exhibit behind it.

2.2 Mouse Interaction

Looking around is implemented with two cooperating input paths so the player is never left with an unresponsive camera:

- **Pointer-lock look** — once the canvas requests the browser's Pointer Lock API, raw `mousemove` deltas rotate yaw and pitch directly, with the very first post-lock event discarded (it otherwise carries a large jump from wherever the OS cursor happened to be) and every delta clamped to ± 160 px so a sudden spike cannot whip the view.
- **Drag-to-look fallback** — whenever the lock is not held (denied by the browser, or the player pressed `Esc`), a pointer-down + pointer-move + pointer-up sequence is tracked instead: dragging turns the camera by the same yaw/pitch update, while a short, low-movement drag is reinterpreted as a plain click so short taps still trigger exhibit interaction.
- **Click-to-interact** — a left click while a target is in range (and not a drag) calls the same `activate()` path as pressing `E`.
- **Mouse-wheel zoom** — in eagle-eye mode only, the wheel adjusts the orbit distance (clamped between 3.2 and 16 units) and is eased toward the target distance every frame instead of snapping.

3. Design and Aesthetic Aspects

3.1 Visual Elements

Every surface in the hall uses a physically-based material (mostly `MeshStandardMaterial`, with `MeshPhysicalMaterial` for the glass entrance doors) so it reacts correctly to the room's lights instead of being flat-shaded:

- **Procedural + photographic textures** — the floor uses a cool polished-terrazzo canvas gradient (roughness 0.25, slight metalness for specular highlights), walls and ceiling use a rougher matte canvas texture (roughness 0.9–0.95) so they stay visually “quiet” behind the exhibits, and wood/marble materials dress the pedestals, door frame and column bases. Posters, plaques and banners layer real Wikimedia Commons photographs (Santiago Bernabéu stadium, the UEFA Champions League trophy, Cristiano Ronaldo) over the same canvases with a gold multiply/screen tint so they match the gallery's warm palette.
- **Lighting model** — `MeshStandardMaterial` (a Physically-Based, metallic/roughness Phong-successor model) is used throughout, so gold trim (metalness 0.85–0.95, roughness 0.28–0.3) reads as burnished metal, marble/plaster stays diffuse (roughness 0.9+), and the trophies (metalness ~ 0.9) pick up the baked PMREM studio reflections described in §1.2. Renderer-side, `PCFSoftShadowMap` gives soft-edged contact shadows and `ACESFilmicToneMapping` (exposure 1.12) keeps the bright accent spotlights from blowing out highlights.
- **Colour and atmosphere** — the scene background and `THREE.Fog` are set to a near-black navy (`#0a0c12`), so distant walls fade softly instead of hard-clipping at the far plane, reinforcing the sense of a large, dim exhibition hall punctuated by warm spotlighting.
- **Cloth banners** — sixteen club banners (in alternating white and navy “way kit” colourways) hang from procedural gold rods around the hall, each displaying a hand-drawn Real Madrid crest (interlaced RMCF

monogram, gold crown, mulberry band) and animate continuously via the custom shader described in §1.3/§4.2.

- **The Cristiano Ronaldo statue** — rendered in a single weathered-stone material combining a procedurally generated bump map (random granular noise + chiselled radial facets) with a marble-veined albedo map, so the whole re-posed figure reads as one carved monument rather than a rigged character.

3.2 Snapshots

The following screenshots, captured directly from the running application, illustrate the main features described above.

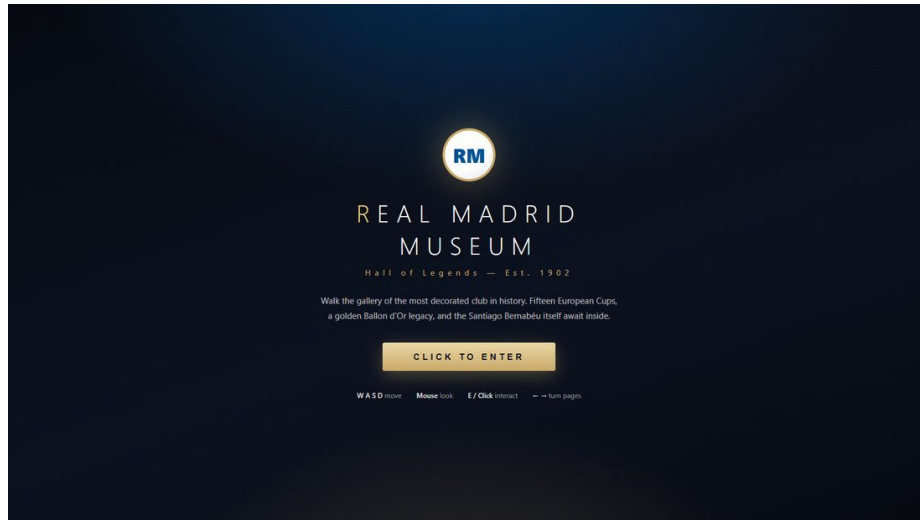


Fig. 1 — Splash / entry screen with control summary.

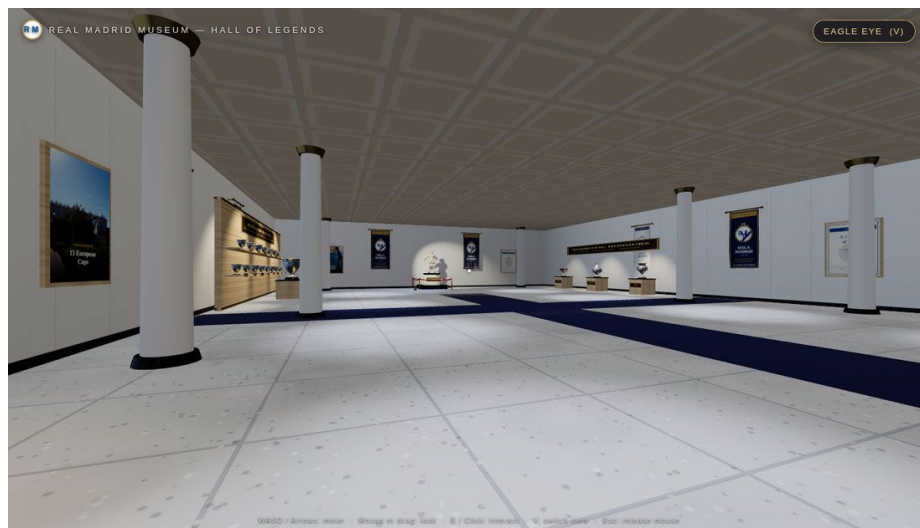


Fig. 2 — Wide eagle-eye view of the Hall of Legends: entrance, UCL wall, columns, carpet cross and posters.



Fig. 3 — UCL Trophy Wall exhibit: fifteen lathed European Cup replicas with year plaques.



Fig. 4 — Domestic Honours exhibit: the Copa del Rey trophy on its wooden pedestal.

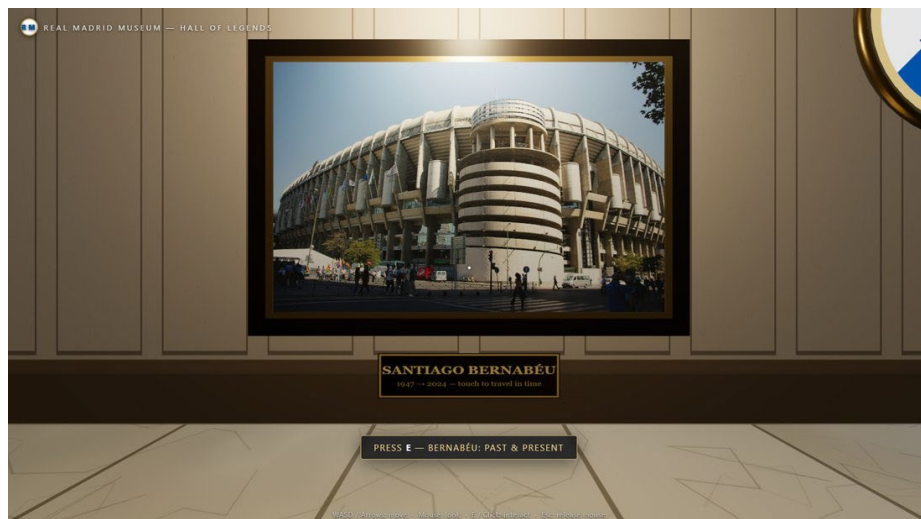


Fig. 5 — The Santiago Bernabéu painting exhibit, lit by its own picture light.

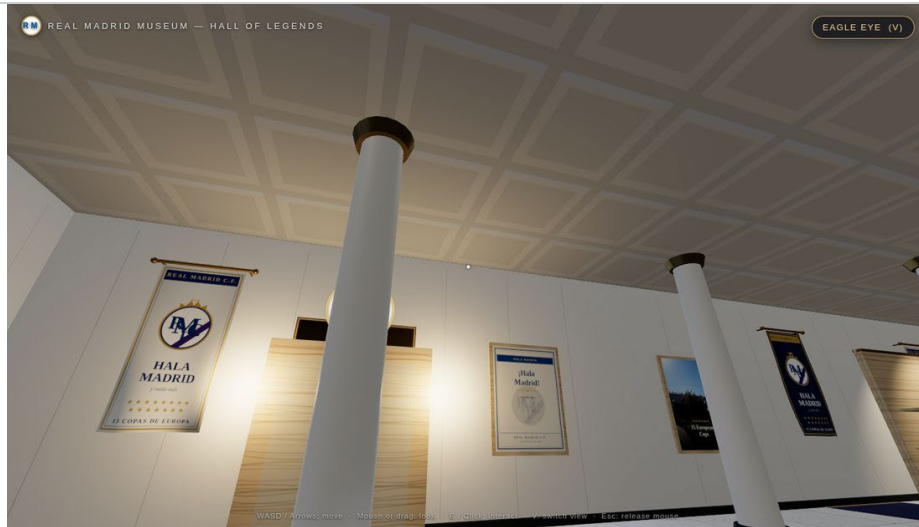


Fig. 6 — Animated cloth banners along the north wall, driven by the custom vertex shader.

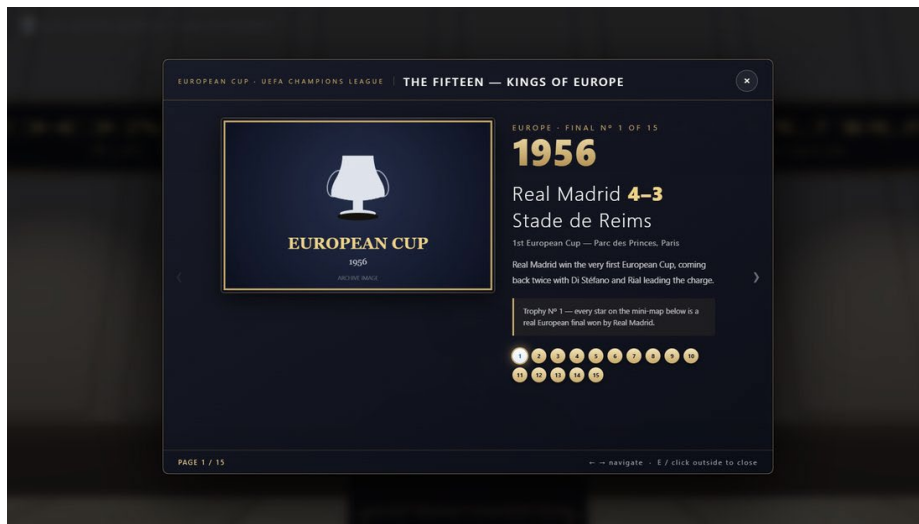


Fig. 7 — The paginated magazine overlay opened on the UCL Trophy Room (Page 1 of 15).

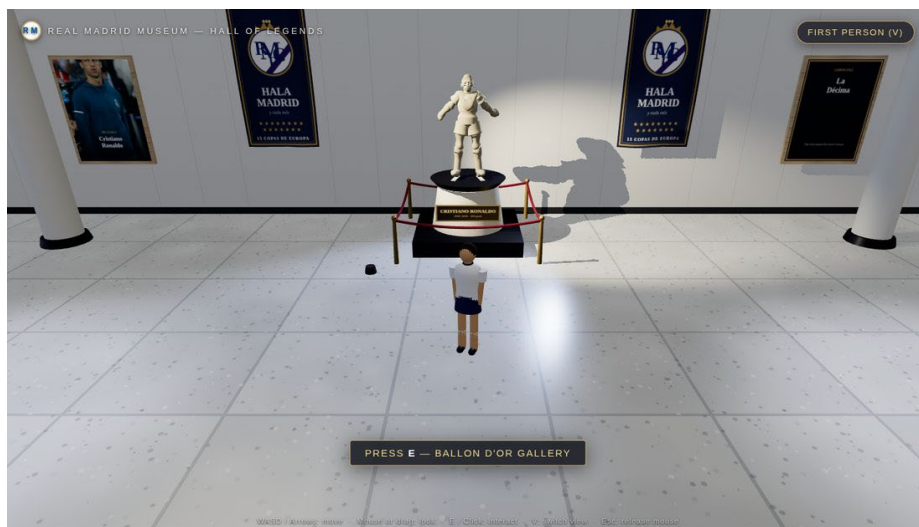


Fig. 8 — Eagle-eye camera showing the animated visitor avatar approaching the Cristiano Ronaldo statue.

4. Code Implementation

4.1 Software Platform

The project is a small, dependency-light static web application:

- **Languages** — HTML5, CSS3, and modern JavaScript (ES modules, top-level `await`).
- **Graphics** — Three.js `r0.182` for the WebGL scene graph, cameras, materials, lights and shadow mapping; the `GLTFLoader` and `BufferGeometryUtils` addons from `three/addons` for loading and welding the statue's glTF models; raw GLSL injected through `onBeforeCompile` for the cloth shader.
- **Runtime / tooling** — a minimal Node.js static file server (`server.js`, started with `npm start`) serves the app over HTTP so ES-module imports and texture/model fetches are not blocked by the `file://` origin restriction; `package.json` declares Three.js as the sole runtime dependency.
- **Assets** — Real photographs are sourced from Wikimedia Commons via `tools/download_assets.py` into `assets/img/`, indexed by `assets/manifest.json`; the statue's base body/head come from the standard three.js sample glTF assets (`Xbot.glb`, `LeePerrySmith.glb`) under `assets/models/`.
- **Testing / QA** — `tools/selftest.js` and `tools/collisiontest.js` drive the scene headlessly through the debug hook `window.__museum.step()` (via Playwright) so interaction and collision behaviour can be verified deterministically, independent of real frame timing; `tools/shots.js` automates the gallery screenshots used in §3.2; `tools/check_shots.py` and `tools/face_crop.py` support image QA for the statue's face texture.

4.2 Custom Shaders

The waving club banners (`src/world/flags.js`) are the project's hand-written GLSL. Rather than replacing `MeshStandardMaterial` outright (which would lose its PBR lighting response), the shader is **patched into** the material via Three.js's `onBeforeCompile` hook, so the cloth still receives the museum's lights and casts/receives shadows like any other mesh:

- **Vertex displacement** — a GLSL function `wave(uv)` sums two travelling sine waves of different spatial frequency, phase speed and per-banner seed, then scales the result by a “slack” falloff `pow(1.0 - uv.y, 1.35)` (nearly rigid at the top rod, loosest at the hem) and an “edge” falloff based on horizontal distance from centre — producing a believable hanging-cloth silhouette rather than a uniform ripple.
- **Analytic normals** — the shader also computes the partial derivatives of the same wave expression with respect to `u` and `v` by hand and rebuilds the vertex normal from them (replacing `#include <beginnormal_vertex>`), instead of relying on flat per-face normals — this is what makes the fabric's specular highlight move convincingly with the ripple.
- **Uniform-driven animation** — a single `uTime` uniform (advanced once per frame from the main render loop) drives every banner; a per-instance `uSeed` uniform offsets the wave phase per banner so the sixteen flags do not all ripple in lock-step.
- **Hem sway** — the vertex's world-Y position is additionally pulled slightly toward the viewer as the wave's local displacement magnitude grows (“the hem swings toward the viewer as it lifts”), adding a secondary, physically-motivated degree of motion beyond the primary wave.

Beyond this hand-written vertex shader, the project also leans on Three.js's built-in physically based shading pipeline (`MeshStandardMaterial`/`MeshPhysicalMaterial`) for every other surface, and on 2-D canvas “shading” (gradients, radial noise, multiply/screen blend compositing) to fabricate texture maps — this is the procedural texture-generation approach used throughout `TextureLibrary`.

4.3 Project Feature Table

#	Feature	Status
---	---------	--------

1	3D scene: room, walls, ceiling, columns, carpet, entrance doors	Implemented
2	Procedural + photo-based PBR texturing (floor, wall, wood, marble, crests)	Implemented
3	Perspective camera with two modes (first-person / eagle-eye orbit)	Implemented
4	Keyboard movement (WASD / Arrow keys) with camera-relative direction	Implemented
5	Mouse-look via Pointer Lock API, with click-and-drag fallback	Implemented
6	AABB collision detection & response (walls, columns, exhibits, statue)	Implemented
7	Dynamic lighting: hemisphere, ambient, spot and point lights + shadow maps	Implemented
8	Custom GLSL vertex shader for animated cloth (waving banners)	Implemented
9	Procedurally lathed/extruded trophy models (UCL, LaLiga, Copa, Supercopa)	Implemented
10	Cristiano Ronaldo statue built and re-posed from real glTF rigs	Implemented
11	Interactive exhibits (press E) opening a paged magazine overlay	Implemented
12	Historical data pages (15 UCL finals, Ballon d'Or, domestic honours)	Implemented
13	Animated eagle-eye visitor avatar with walk-cycle & turn-to-face	Implemented
14	Old → new Bernabéu painting cross-fade on proximity interaction	Implemented
15	Environment reflection map (PMREM) for metallic trophies/gold trim	Implemented
16	ACES Filmic tone mapping and scene fog for depth cueing	Implemented
17	Window-resize responsive rendering	Implemented
18	Automated headless tests & screenshot capture tooling (Playwright)	Implemented
19	Sprint (Shift) movement and footstep view-bob	Implemented
20	Mobile / touch-specific control scheme	Not Implemented

Table 01: Project Feature Table

5. Contribution

Both members collaborated closely throughout the project (shared design decisions, joint debugging and testing), with primary ownership split roughly as follows:

Syed Abir Hossain (20220104013)

- Core rendering setup: renderer/scene/camera configuration, tone mapping, fog, and the procedural environment map (PMREM) used for reflections.
- Player systems: first-person / eagle-eye camera blending, pointer-lock and drag-to-look input, keyboard movement, and the axis-separated collision detection & resolution logic.
- Lighting design across the hall and per-exhibit accent spotlights, and the base TextureLibrary (procedural floor/wall/ceiling/wood/marble textures and photo-overlay pipeline).
- The custom cloth vertex shader for the waving banners.

Alim Bin Yeasin (20220104016)

- Exhibit and trophy modelling: the lathed/extruded UCL, LaLiga, Copa del Rey and Supercopa trophies, pedestals and wall displays.
- The Cristiano Ronaldo statue: glTF import, skeletal re-posing (aim/twist helpers), procedural kit geometry, and the stone material/texture treatment.
- UI/UX: the magazine overlay (open/close, page navigation) and the historical content pages (UCL finals, Ballon d'Or, domestic honours data).
- Testing and asset tooling: the headless self-test and collision-test scripts, screenshot automation, and asset-download utilities.